# HOSPITAL EQUIPMENT WIRING USING KRUSKAL'S ALGORITHM

*Modified Project: Edge Weight Update, Running Total Cost, and Disconnected Graph Detection*

## 1. Problem Statement

Develop a C++ program to represent hospital equipment units as vertices and possible wiring connections as weighted edges. Apply Kruskal's algorithm to select minimum-cost wiring connections. The program must allow an edge weight to be updated and Kruskal's algorithm to be run again, display each selected edge with its weight and running total cost, identify edges rejected because they create cycles, and detect whether the graph is disconnected and a spanning tree can be formed.

## 2. Solution

Hospital equipment units are represented as vertices, and possible wiring connections are weighted edges. The program sorts edges in ascending order of weight and applies Kruskal's algorithm to select the lowest-cost edges without creating cycles. A Disjoint Set Union (DSU) data structure checks whether the endpoints of an edge are already connected. For every selected edge, the program displays its weight and updates the running total cost. If V - 1 edges are selected, a spanning tree exists; otherwise, the graph is disconnected. The user can update an existing edge weight, after which the edges are sorted and Kruskal's algorithm is applied again.

## 3. Objectives

1. To represent hospital equipment units as vertices in a weighted graph.
2. To find minimum-cost wiring connections using Kruskal's algorithm.
3. To update an edge weight and recalculate the minimum spanning tree.
4. To display each selected edge with its weight and running total cost.
5. To identify edges rejected because they create cycles.
6. To detect a disconnected graph and report whether a spanning tree can be formed.

## 4. Algorithm

7. Start the program.
8. Input the number of equipment units and possible wiring connections.
9. Enter equipment names and each connection's source, destination, and weight.
10. Sort all edges in ascending order of weight.
11. Initialize DSU and set the running total cost and selected-edge count to zero.
12. Process the edges using Kruskal's algorithm.
13. If an edge connects different sets, select it, merge the sets, and add its weight to the running total.
14. If both endpoints are already in the same set, reject the edge because it creates a cycle.
15. Display each selected edge, its weight, and the running total cost.
16. If V - 1 edges were selected, report that a spanning tree exists; otherwise, report that the graph is disconnected.
17. Ask whether the user wants to update an edge weight.
18. If yes, enter the edge and new weight, update the edge if it exists, sort the edges again, and rerun Kruskal's algorithm.

19. If no update is requested, end the program.

## 5. Flowchart

